Email Authentication Demo - README

Email Authentication Demo SPF + DKIM Validation with Local SMTP Capture

This project is a self-contained demonstration of how SPF and DKIM help differentiate legitimate emails from spoofed ones. It includes:

- A local SMTP server that captures raw .eml files - A multi-client simulation (legitimate sender vs. spoofed sender) - SPF policy simulation - DKIM signing and verification - Tools for analyzing message headers and verifying DKIM on saved emails

The goal is to show how email origin authentication works in practice and how spoofed messages can be detected.

Project Structure

- generate_client_keys.py – Generates DKIM private/public keys for each domain - multi_client_demo.py – Main demo: Sends messages, SPF+DKIM checks - smtp_server.py – Local SMTP server capturing emails - spf_check.py – SPF policy simulation - verify_local_dkim.py – DKIM verification for saved .eml files - header_analysis.py – Analyzes From/To/Subject/DKIM-Signature - requirements.txt – Dependencies - keys/ – Generated at runtime – DKIM keypairs - messages/ – Captured .eml message files

Getting Started

1. Install Dependencies

It’s strongly recommended to use a virtual environment.

python -m venv .venv source .venv/bin/activate # macOS/Linux # OR .\.venv\Scripts■ctivate # Windows PowerShell

Then install:

pip install -r requirements.txt

This installs: - aiosmtpd (local SMTP server) - dkimpy (DKIM signing/verification) - cryptography (RSA key generation)

Running the Demo

Step 1 — Start the SMTP Capture Server

In Terminal #1:

python smtp_server.py

This starts a server on 127.0.0.1:1025.

Every message sent to this SMTP server is stored in the messages/ folder as a raw .eml file.

You should see output like:

SMTP server listening on 127.0.0.1:1025 Saving messages to ./messages/

Step 2 — Generate DKIM Keys

In Terminal #2:

python generate_client_keys.py

This creates directories like:

keys/ bankofamerica.test/ default.private default.public du-students.test/ default.private default.public

Each client/domain gets its own DKIM keypair.

Step 3 — Run the Simulation

Still in Terminal #2:

python multi_client_demo.py

This script:

- Loads client configurations (legit vs. spoofed) - Simulates SPF for each domain - DKIM-signs legitimate emails - Sends both signed and unsigned messages to the SMTP server - Performs DKIM verification: - in memory, before sending - after saving, using the .eml file

It finishes by classifying each email as:

- Trusted - Suspicious / Rejected

Viewing Captured Emails

All raw emails are stored in:

./messages/msg_.eml

One example is a DKIM-signed message (legitimate): - Includes a full DKIM-Signature header - Sent from alerts@bankofamerica.test to victim@local.test

Another example is an unsigned spoofed message: - No DKIM-Signature header - Sent from financial-aid@du-students.test to victim@local.test

Both files match exactly what the simulator generates.

Analyzing Messages

1. Check Headers

python header_analysis.py messages/.eml

Reports:

- From - To - Subject - Whether a DKIM-Signature is present

2. Verify DKIM Using Local Public Key

python verify_local_dkim.py messages/.eml keys/bankofamerica.test/default.public

This gives a PASS/FAIL result using dkimpy.

High-Level Architecture

Clients: - Legitimate Sender: bankofamerica.test - Spoof Sender: du-students.test

Flow:

- generate_client_keys.py creates DKIM keys. - multi_client_demo.py builds messages, invokes SPF + DKIM, and sends via SMTP. - spf_check.py simulates domain/IP authorization. - smtp_server.py receives emails on 127.0.0.1:1025 and writes .eml files. - header_analysis.py inspects basic headers. - verify_local_dkim.py verifies DKIM signatures using the public key.

Component Summary

1. generate_client_keys.py Creates a DKIM RSA private/public keypair per domain.

2. smtp_server.py A lightweight SMTP server (via aiosmtpd) running on port 1025. Every incoming message is stored in raw bytes and becomes a timestamped .eml file.

3. multi_client_demo.py Simulates multiple clients:

Client Type SPF DKIM Expected Outcome Legitimate PASS Signed Trusted Spoofed FAIL None Suspicious / Rejected

4. spf_check.py Implements a simple SPF rule using a domain to allowed IP mapping.

5. header_analysis.py Reads an .eml file and reports basic header presence.

6. verify_local_dkim.py Validates DKIM signatures using the .eml file and the matching domain's public key.

Sample Output

Example (simplified):

[bankofamerica.test] SPF PASS [bankofamerica.test] DKIM verified: True Message saved: messages/msg_2025...eml Overall Status: TRUSTED

[du-students.test] SPF FAIL [du-students.test] DKIM verified: False (no signature) Overall Status: SUSPICIOUS/REJECTED